

Detecting Tuberculosis From Ziehl-Neelsen Stained Sputum Smears

Nicholas Nolen

March 27, 2026

Abstract

Tuberculosis is one of the world's most infectious and deadly diseases. Tuberculosis affects about 25% of the world's population and is responsible for roughly 1.2 million deaths each year. In this work I investigate using a CNN model to assist health officials in the detection of Tuberculosis on sputum slides stained with the Ziehl-Neelsen microscopy technique. Through my methods I achieved a 0.89 positive prediction rate and a 0.41 false-positive prediction rate.

1. Introduction

According to the World Health Organization (WHO) approximately a quarter of the world's population is currently infected with Tuberculosis. In addition, the disease has been shown to be much more likely to have extreme adverse health impacts and be much harder to cure for those living in poverty. The best way to prevent the spread of Tuberculosis in these places is being able to diagnose the disease early, however most diagnostic tools are very expensive and the people living in these communities do not have access to them.

The current most widespread diagnostic tool for Tuberculosis in impoverished areas is sputum smear microscopy. This is a diagnostic technique where a trained technician examines a sputum smear from a patient under a microscope and looks for Tuberculosis bacteria. This method is time-consuming and has the potential to overlook between 33%-55% of cases.

Because of the low detection rate of the manual method, I have created a CNN image classification model which can be used to assist lab technicians by finding (244×244) pixel image crops from a microscope viewpoint and sort the crops by their likelihood of containing Tuberculosis Bacilli. This proposed method has the potential to greatly speed up the lengthy and error prone process of manually detecting Tuberculosis Bacilli.

2. Related Work

A description of similar work in the area of using machine learning models for Tuberculosis Bacilli detection on Ziehl-Neelsen sputum smears:

Automatic bright-field smear microscopy for diagnosis of pulmonary Tuberculosis: The authors of this paper also used a CNN to detect Tuberculosis. They evaluated multiple architectures and even used an encoder-decoder architecture as opposed to a generic CNN architecture, finding it to have better results.

While the authors of this paper had good success in training their model, I believe that my proposed methods are improvements in that their crop size was much larger than mine at (400×400) rather than (244×244) . Further, they only used one dataset for their images, choosing to strengthen their model through augmentation, while I plan to use multiple distinct datasets in addition to augmenting my training images.

Pulmonary Tuberculosis Diagnosis Using an Intelligent Microscopy Scanner and Image Recognition Model for Improved Acid-Fast Bacilli Detection in Smears: The authors of this paper were the ones who constructed the dataset the Wellgen Medical Subset was based off of. This study constructed a CNN model using 100,000 sputum slides. Due to the large amount of input, their model was able to perform very well, but because they used entire sputum slides as input there is potential that the model overfit some features.

My method can potentially be an improvement on their method because, rather than use the entire slide as input, I extract crops which are easier for the CNN model to pull features out of and make inferences on. This reduces the size of my model and the amount of training data necessary for me to get adequate results.

Automated Mycobacterium Tuberculosis Detection in Multivariant Digitized Ziehl-Neelsen Staining Using Faster R-CNN Method: This is a paper outlines training a Faster R-CNN model on the TBDB(Trilaksana et al., 2020) dataset. Their methods retained the (612×480) image size for training their R-CNN and in addition augmented saturation, color, brightness, stretched the images, and added

blur.

There are definitely merits to the augmentations done by this team, however I believe that my model is an improvement over their methods because my crop size is smaller, meaning my model is able to pick up on features of the small Tuberculosis Bacilli quicker than theirs. Further, I use more diverse datasets, meaning my model is likely more robust to variations in lab equipment and would therefore perform better in the real world.

3. Methodology

The main approach I used to train the CNN is to crop the dataset images from their starting resolution to a resolution of (244×244) pixels around labeled Tuberculosis Bacilli. I then use this model trained on (244×244) sized images of Tuberculosis Bacilli to detect the presence of Tuberculosis Bacilli in (244×244) crops of any slide.

I am generating the crops using the following dataset and methods:

ODS3 dataset: The Original Dataset 3(ODS3)(Costa et al., 2025) is a dataset which was compiled using 3,280 extended depth focus bitmap images. If there are Tuberculosis Bacilli present on the image there is a corresponding CSV with “b” or “bc” relating to a detected bacillus or bacillus cluster. I used these CSVs to create (244×244) crops of the images which can be used to train the CNN model at a closer resolution to better capture the features of the Tuberculosis bacteria. Creating these crops from the positively labeled Tuberculosis Bacilli resulted in 10,897 positive images. I then compiled a similar number of negative (244×244) image crops by cropping (244×244) images from slides without corresponding annotation files. I did this to get 4 crops per image which resulted in 9,726 negative crops.

TBDB dataset: The Tuberculosis Database (TBDB)(Trilaksana et al., 2020) contains 600 (612×480) resolution images of Tuberculosis sputum smears along with XML files containing bounding boxes detailing the location of the Tuberculosis Bacilli and Debris on the image. To make this data compatible with my model architecture, I cropped out (244×244) segments from these images where there was a Tuberculosis Bacillus labeled as present. This resulted in 1,482 positive image crops. In addition, to generate negative class images I made crops around the annotation locations which were labeled as Debris.

As the crops generated from this dataset are slightly larger than the ones generated from the ODS3 dataset, it is useful in training the model to detect Tuberculosis Bacilli of more varying sizes.

Testing via Wellgen Medical Subset: The Wellgen Medical dataset(Lin & Medical, 2024) is a 1000 image subset of a large dataset of (2448×2048) Ziehl-Neelsen sputum smear slides. The slides in this dataset are labeled positive and negative, but none of the positive slides have bounds around the locations of the Bacilli. Because of this I cannot use this dataset to train the CNN model. However, I can use this dataset to get an idea of how well my model performs on data it hasn’t been trained on.

Specifically, to test models using the images in this dataset I separate the (2448×2048) images into 80 (the maximum number possible) (244×244) crops. I then have the model being tested predict the probability that Tuberculosis Bacilli are present in each of the crops and if the probability is greater than a threshold I count the slide as being positive for Tuberculosis. I run this for the 100 positive slides in the dataset and 100 of the negative slides in the dataset and then divide the number of positive labels for either slide by the number of slides tested (100 slides). This results in a number representing the percent of slides labeled above the threshold out of the total number of slides.

The number of positive prediction labels can be called a positive prediction rate and the rate of the number of negative slides labeled positive can be interpreted as the false-positive prediction rate.

Ultimately this model is being trained to assist lab technicians in diagnosing Tuberculosis, so the average prediction is not as important as how well the model is able to sort sputum crops based on their likelihood of containing Tuberculosis. However, the average positive and false-positive predictions are still useful as a metric to how well the model performs to maximize its effectiveness in sorting.

4. Experiments

I began my experiments working with my cropped modification of the ODS3 dataset. My method for selecting weights to save is based off of minimizing Binary Cross Entropy Loss as that appeared to be the best metric for loss for a Binary Classification CNN architecture.

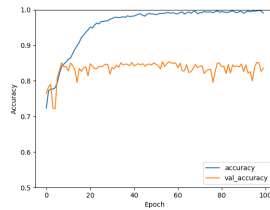
The following is my starting configuration for my CNN model:

Model: "sequential"		
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 3)	96
max_pooling2d (MaxPooling2D)	(None, 16, 16, 3)	0
conv2d_1 (Conv2D)	(None, 32, 16, 3)	16,968
max_pooling2d_1 (MaxPooling2D)	(None, 16, 8, 3)	0
conv2d_2 (Conv2D)	(None, 32, 8, 3)	86,400
flatten (Flatten)	(None, 2688)	0
dense (Dense)	(None, 1)	2,689
Total params: 104,464 (1.01 MB)		
Trainable params: 104,464 (1.01 MB)		
Non-trainable params: 0 (0.00 B)		

Identifying Number of Epochs: First, I wanted to select an appropriate number of epochs to prevent overtraining

and limit the amount of time spent training a model.

Training the CNN for 100 epochs resulted in the following graph:



Based on the divergence of the training accuracy versus the validation accuracy it appears that 20 epochs is sufficient to to achieve the best training results while minimizing the time spent overfitting.

Drafting CNN Architecture to Minimize Loss: Using my initial CNN configuration I achieved a validation accuracy of 0.8264 and a validation loss of 0.4319.

While the accuracy is fairly high, a loss of 0.4319 shows that the model is uncertain about the validity of its predictions.

I increased the number of filters for the first convolutional layer to 64, this would hopefully allow me to capture the shape of the small Bacilli more accurately from the beginning I also lowered the kernel size from (3,3) to (1,1) to try and understand how the size of the kernel affects detection.

This experiment resulted in a validation accuracy of 0.8992 and validation loss of 0.2958. Simply from the change in architecture, both validation accuracy and loss have greatly improved.

To further increase accuracy and decrease loss I tried the model with 128 filters on the first convolutional layer. This change resulted in a validation accuracy of 0.8938 and a validation loss of 0.3129.

Because of the significant increase in training time and slight decrease in accuracy, I then decided to evaluate starting with less features in the first few layers and attempt to increase features as I downsample.

Model: "sequential"		
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 64, 64, 1)	64
max_pooling2d (MaxPooling2D)	(None, 32, 32, 1)	0
conv2d_1 (Conv2D)	(None, 64, 32, 3)	6,048
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 3)	0
conv2d_2 (Conv2D)	(None, 64, 16, 3)	11,520
flatten (Flatten)	(None, 4096)	0
dense (Dense)	(None, 1)	4096

This architecture resulted in a validation accuracy of 0.8606 and a validation loss of 0.3823, which are both a downgrade from the previous method.

These results led me to believe that my model was not com-

plex enough and therefore could not capture an adequate hypothesis which would allow it to fit the data well. Because of this I decided to start at 64 filters rather than 32 and added another convolutional layer and max pooling layer both with 256 filters.

This combination resulted in a validation accuracy of 0.9092 and a validation loss of 0.2901 which is the best accuracy and smallest loss I have gotten yet while still having a reasonable training time.

Optimizing Architecture Based on Test Set: While it is acceptable to use the validation set to search for an appropriate group of hypothesis classes, which in this case it would be the structure of the network, it is not adequate to use the validation set to estimate the model's overall performance.

To approximate the classification power of the model I will be using the previously described test set of Ziehl-Neelsen slides which contains images that none of the models were trained on. The accuracy threshold I will be using to evaluate my probabilities is 0.80

With my latest model I achieved a positive prediction rate of 0.53 and a false-positive prediction rate of 0.45.

These numbers show that the model is doing a very poor job at correctly detect Tuberculosis bacterium and major improvements still need to be made before the model is viable.

The high testing accuracy and low real-world accuracy of my model lead me to believe that my data was overfitting. So, because simpler models are less prone to overfitting I reduced the amount of filters present in my model. However, at the same time I increased the number of layers believing that with the additions of more layers my model will still able to fit the data well.

This following architecture achieved a positive prediction rate of 0.93 and a false-positive prediction rate of 0.44:

Model: "sequential"		
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 64, 64, 1)	64
max_pooling2d (MaxPooling2D)	(None, 32, 32, 1)	0
conv2d_1 (Conv2D)	(None, 64, 32, 3)	6,048
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 3)	0
conv2d_2 (Conv2D)	(None, 64, 16, 3)	11,520
max_pooling2d_2 (MaxPooling2D)	(None, 8, 8, 3)	0
conv2d_3 (Conv2D)	(None, 64, 8, 3)	11,520
max_pooling2d_3 (MaxPooling2D)	(None, 4, 4, 3)	0
conv2d_4 (Conv2D)	(None, 64, 4, 3)	11,520
flatten (Flatten)	(None, 16384)	0
dense (Dense)	(None, 1)	16,384

The results are beginning to look favorable, to push my model even further I decided to increase the number of negative examples in my modified ODS3 training set so it was closer to the number of positive values.

This modification to my training set resulted in a positive prediction rate of 0.98 and a false-positive prediction rate of

0.68. This large bias to the positive led me to simplify my model even more, resulting in the following configuration:

Model: "sequential"		
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 252, 252, 3)	132
max_pooling2d (MaxPooling2D)	(None, 126, 126, 3)	0
conv2d_1 (Conv2D)	(None, 110, 110, 16)	592
max_pooling2d_1 (MaxPooling2D)	(None, 55, 55, 16)	0
conv2d_2 (Conv2D)	(None, 57, 57, 16)	2,328
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 16)	0
conv2d_3 (Conv2D)	(None, 28, 28, 32)	4,640
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 32)	0
flatten (Flatten)	(None, 5488)	0
dense (Dense)	(None, 1)	5,489

This configuration resulted in a positive prediction rate of 0.89 and a false-positive prediction rate of 0.41, much more distributed from each other, but still showing that the model is having a hard time accurately detecting negative classes.

To further improve my model I will randomize my crops every 5 epochs, this will prevent overfitting based on location and will help the model learn features like the shape of the Bacilli and size.

Also, I will augment the dataset with the TBDB dataset to help the model learn variations in size of the bacteria, as the Bacilli in the TBDB appear slightly larger than they do in the ODS3 dataset.

Finally, I will add images with varying saturation, color, brightness, stretched the images, blur similar to the process that Trilaksana et al. underwent with their model to further improve my model's generalization power.

5. Discussion and Analysis

Discussion: Detecting Tuberculosis via Ziehl-Neelsen microscopy is the cheapest and most accessible Tuberculosis diagnostic technique but very time-consuming and prone to false negatives. Because of this, a tool that accurately flags potential Tuberculosis presence which can be reviewed by a technician is highly useful. Such a tool could be operated relatively inexpensively but greatly increase the rate of Tuberculosis detection and the speed of diagnosis by displaying crops with the highest probability of Tuberculosis presence to help a technician narrow down potential sites of Tuberculosis activity quicker than if they were manually observing a sputum slide.

Analysis: A 0.89 positive prediction rate is quite ideal. This rate shows that there is a high likelihood that crops with Tuberculosis Bacilli will be preferred by the model to images without Bacilli. Conversely, a 0.41 false-positive prediction rate still needs to be improved. This result implies that almost half of the crops tested by the model will be incorrectly labeled with a high likelihood that they contain Tuberculosis Bacilli.

6. Conclusion

The problem of detecting Tuberculosis is difficult because the places where it is most prevalent also lack adequate healthcare funding to do something about it. The most accessible diagnostic tool in the poorer regions where Tuberculosis is highly infectious is Ziehl-Neelsen stained slide microscopy. This technique is cheap and therefore accessible, but requires trained technicians to spend hours looking through microscope lenses to diagnose a single patient. The classification system proposed in this paper would be able to greatly speed up the rate at which a technician is able to classify Tuberculosis Bacilli by presenting areas of a view-field to the technician where there is the greatest likelihood of Tuberculosis being present.

References

- Costa, M. G. F., Serrão, M. K. M., Fujimoto, L. B. M., Ogusku, M. M., Romero, F. P., Romaguera, L. V., Pinto, K. M. B., López, Y. P., Binatti, F., and Filho, C. F. F. C. ODS3 - Dataset of smear microscopy images labelled to evaluate methods for Tuberculosis diagnosis. 2 2025. doi: 10.25452/figshare.plus.27209355.v1. URL https://plus.figshare.com/articles/dataset/_b_ODS3_-_Dataset_of_smear_microscopy_images_labelled_to_evaluate_methods_for_Tuberculosis_diagnosis_b_/27209355.
- Lin, E. and Medical, W. Detecting mycobacterium tuberculosis bacilli from tb smear images, 2024. URL <https://doi.org/10.17632/yhybmfmrms.1>.
- Trilaksana, H., Dwimudyari, G. N., Agoes, A. S., and Widhyatmoko, D. B. Sputum smear images database: A resource for deep learning study based to detect bacilli for tb diagnose. *AIP Conference Proceedings*, 2314(1): 040013, 12 2020. ISSN 0094-243X. doi: 10.1063/5.0036388. URL <https://doi.org/10.1063/5.0036388>.